

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	RAJESH KUAMR V	Reg. No.:	192511389
Section / Batch:	2025	Date:	16/07/2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical

explanatio
n are
mandator
y

	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Q1. A cloud system processes backup data using the recurrence $T(n) = T(n/2) + n$. Recall recurrence solving techniques, explain how the function grows, and determine the asymptotic time complexity using an appropriate method. Also analyze the space complexity of the recursive calls.

Q2. A digital library system uses a divide-and-conquer strategy to locate books by repeatedly halving the search space. Explain how this reduction in input size affects the number of operations required. Determine the time complexity of the algorithm using asymptotic notation and justify your reasoning. Also analyze the space complexity, particularly in recursive implementations. Interpret how this approach improves efficiency compared to linear search for large-scale library systems.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem

Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Cloud Backup Analysis System

A cloud system processes backup data using the recurrence $T(n) = T(n/2) + n$. Recall recurrence solving techniques, explain how the function grows, and determine the asymptotic time complexity using an appropriate method. Also analyze the space complexity of the recursive calls.

Step 1: Understanding the Recurrence

The given recurrence relation is:

$$T(n) = T(n/2) + n, \quad T(1) = O(1)$$

This recurrence describes an algorithm that, at every stage, divides the backup data into half (subproblem size $n/2$) and performs $O(n)$ work outside the recursive call — for example, scanning, comparing, or merging n data blocks before/after the recursive halving step.

Step 2: Identify the Master Theorem Parameters

The general form of a divide-and-conquer recurrence is:

$$T(n) = a \cdot T(n/b) + f(n)$$

Comparing with $T(n) = T(n/2) + n$, we identify:

- $a = 1$ (number of recursive sub-calls)
- $b = 2$ (factor by which input size is divided)
- $f(n) = n$ (work done outside the recursive call, i.e. combining/scanning cost)

Step 3: Compute the Watershed Function

Master Theorem compares $f(n)$ with $n^{\log_b a}$:

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1$$

So we compare $f(n) = n$ with $n^0 = 1$.

Step 4: Apply Master Theorem Case

Since $f(n) = n$ grows polynomially faster than $n^{\log_b a} = 1$, i.e. $f(n) = \Omega(n^{0+\epsilon})$ for $\epsilon = 1$, and it also satisfies the regularity condition ($a \cdot f(n/b) = 1 \cdot (n/2) \leq c \cdot f(n)$ for some $c < 1$, e.g. $c = 1/2$), this falls under Master Theorem Case 3 (the 'unbalanced' case where the combining cost dominates).

By Case 3 of the Master Theorem:

$$T(n) = \Theta(f(n)) = \Theta(n)$$

Step 5: Verification by Recursion Tree / Substitution Method

Recursion tree method — expanding the recurrence level by level:

- Level 0: cost = n
- Level 1: cost = $n/2$
- Level 2: cost = $n/4$
- ... continuing until the subproblem size reaches 1, after $\log_2 n$ levels

Total cost = $n + n/2 + n/4 + \dots + 1$, which is a geometric series with ratio $1/2$.

$$Sum = n \cdot (1 + 1/2 + 1/4 + \dots) \approx 2n = \Theta(n)$$

This confirms the result obtained using the Master Theorem: the total work is dominated by the very first level of recursion (the top of the tree), since the series converges quickly to a constant multiple of n .

Substitution method (verification): Assume $T(n) \leq cn$ for some constant c . Then:

$$T(n) = T(n/2) + n \leq c(n/2) + n = n(c/2 + 1) \leq cn, \text{ for } c \geq 2$$

This holds by induction, confirming $T(n) = O(n)$. Combined with the trivial lower bound $T(n) = \Omega(n)$ (since $f(n) = n$ is done at the top level alone), we get $T(n) = \Theta(n)$.

Step 6: How the Function Grows

As n increases, $T(n)$ grows linearly — doubling the size of the backup data roughly doubles the total processing time. This is because although the recursion halves the problem repeatedly, the dominant cost is the $O(n)$ work performed at the very first (outermost) call; the work at deeper levels shrinks geometrically and contributes only a small constant fraction more to the total.

Step 7: Space Complexity Analysis

Each recursive call to $T(n/2)$ creates a new stack frame that must be retained until the call returns (LIFO order), since this is a single-branch ($a = 1$) linear recursion, not a tree with multiple simultaneous branches.

The recursion depth is the number of times n can be halved before reaching the base case:

$$\text{Depth} = \log_2(n)$$

Since each stack frame uses $O(1)$ auxiliary space (aside from the recursive call itself), the total auxiliary space complexity of the recursive call stack is:

$$S(n) = O(\log n)$$

If the algorithm also needs to store the input data itself (e.g., the n backup blocks) at each level for combining/merging, the space could rise to $O(n)$; but for the recursive call-stack space alone, it is $O(\log n)$.

Conclusion

Time Complexity: $T(n) = \Theta(n)$, obtained using Master Theorem Case 3 and verified using the recursion-tree and substitution methods.

Space Complexity: $O(\log n)$ for the recursive call stack, due to the logarithmic recursion depth arising from halving the input at each step.

This means the cloud backup system's processing time scales linearly with the amount of backup data, while the memory used by recursion remains small (logarithmic), making the approach efficient and scalable for large backup volumes.

Q2. Library Search System (50 Marks)

A digital library system uses a divide-and-conquer strategy to locate books by repeatedly halving the search space. Explain how this reduction in input size affects the number of operations required. Determine the time complexity of the algorithm using asymptotic notation and justify your reasoning. Also analyze the space complexity, particularly in recursive implementations. Interpret how this approach improves efficiency compared to linear search for large-scale library systems.

Step 1: Identifying the Algorithm and Its Recurrence

Repeatedly halving the search space to locate a target (a book, by call number/ISBN) in a sorted catalogue is the classic Binary Search strategy. At each step, the algorithm compares the target with the middle element and discards one half of the remaining search space, recursing only into the other half.

The recurrence relation for this divide-and-conquer approach is:

$$T(n) = T(n/2) + O(1)$$

Here $O(1)$ represents the constant-time comparison performed at each step (checking the middle element against the target), and $T(n/2)$ represents the recursive search in the remaining half.

Step 2: Effect of Reducing Input Size

Because the search space is halved at every step, the number of elements under consideration shrinks geometrically:

- After 1 comparison: $n/2$ elements remain
- After 2 comparisons: $n/4$ elements remain
- After k comparisons: $n/2^k$ elements remain

The search terminates when only 1 element remains, i.e. when $n/2^k = 1$, giving $k = \log_2 n$. So the number of operations required grows only logarithmically with the size of the catalogue — doubling the number of books adds just one extra comparison.

Step 3: Determining Time Complexity (Master Theorem)

Comparing $T(n) = T(n/2) + O(1)$ with the general form $T(n) = a \cdot T(n/b) + f(n)$:

- $a = 1$, $b = 2$, $f(n) = O(1) = O(n^0)$

$$n^{(\log_b a)} = n^{(\log_2 1)} = n^0$$

Since $f(n) = \Theta(n^0)$ matches $n^{(\log_b a)} = n^0$ exactly, this satisfies Master Theorem Case 2:

$$T(n) = \Theta(n^{(\log_b a)} \cdot \log n) = \Theta(\log n)$$

Justification via recursion tree: the recursion has exactly one branch per level ($a = 1$), and there are $\log_2 n$ levels (since the problem size is divided by 2 until it reaches 1). Each level does $O(1)$ work, so total work = $O(1) \times \log_2 n = \Theta(\log n)$.

Step 4: Space Complexity Analysis

Recursive implementation: Each recursive call adds one stack frame and is only resolved after its child call returns. The recursion depth equals the number of halving steps, $\log_2 n$, so:

$$S(n) = O(\log n) \text{ (recursive version)}$$

Iterative implementation: If binary search is implemented with a loop instead of recursion, no extra call-stack frames are created beyond a few pointer/index variables, so:

$$S(n) = O(1) \text{ (iterative version)}$$

Thus, the space complexity particularly depends on the implementation style — recursive binary search trades a small amount of stack space ($O(\log n)$) for cleaner code, while the iterative version achieves constant space.

Step 5: Comparison with Linear Search

Aspect	Linear Search	Binary Search (Library System)
Search Space Reduction	Checked one element at a time	Halved at every comparison

Time Complexity	$O(n)$	$O(\log n)$
Precondition	None (works on unsorted data)	Requires sorted catalogue
Space Complexity	$O(1)$	$O(\log n)$ recursive / $O(1)$ iterative
Scalability	Poor for large n	Excellent for large n

Step 6: Interpretation / Efficiency Gain

For a library catalogue with, say, 1,000,000 books, linear search may require up to 1,000,000 comparisons in the worst case, whereas binary search requires only about $\log_2(1,000,000) \approx 20$ comparisons. This dramatic reduction — from a linear to a logarithmic growth rate — means that as the digital library scales up to millions of records, binary search remains extremely fast, while linear search becomes increasingly impractical.

This efficiency comes directly from the divide-and-conquer principle: by discarding half of the remaining candidates at every step instead of examining them one by one, the algorithm reduces the problem size exponentially fast relative to the number of comparisons performed.

Conclusion

Time Complexity: $\Theta(\log n)$, derived using Master Theorem Case 2 and confirmed by the recursion-tree analysis ($\log_2 n$ levels, $O(1)$ work per level).

Space Complexity: $O(\log n)$ for the recursive implementation (due to call-stack depth) and $O(1)$ for the iterative implementation.

Overall, binary search's logarithmic time complexity makes it vastly more efficient than linear search's linear complexity for large-scale library systems, at the small cost of requiring the catalogue to be maintained in sorted order.